

Relazione App KeyWord Spotting

Studente: Roggio Giuseppe – 1000050655

Il progetto che ho scelto di implementare tra quelli proposti è stato il n.1 l'applicazione di riconoscimento vocale. L'app ha lo scopo di analizzare l'audio in tempo reale ed avvisare l'utente tramite log e notifiche quando intercetta parole del suo vocabolario, andando anche a differenziare parole **comuni** da parole **sensibili**.

Funzionamento dell'App

L'applicazione **KeyWord Spotting** è progettata per ascoltare continuamente l'ambiente tramite il microfono del dispositivo mobile e identificare le parole riconosciute dal modello `speech_commands.tflite`. Il flusso di funzionamento è il seguente:

1. **Acquisizione Audio:** L'applicazione cattura i dati audio in tempo reale dal microfono del dispositivo.
2. **Pre-elaborazione:** L'audio grezzo acquisito viene trasformato in un formato (`TensorAudio`) comprensibile dal modello di machine learning. Questa fase include la campionatura, la normalizzazione e l'estrazione di caratteristiche audio rilevanti.
3. **Inferenza:** L'`AudioClassifier`, parte della TensorFlow Lite Task Library, elabora il `TensorAudio` attraverso il modello `speech_commands.tflite`. Il risultato è un output che consiste in una lista di categorie (le parole potenzialmente riconosciute) ciascuna associata a un punteggio di confidenza.
4. **Post-elaborazione e Azione:** L'applicazione analizza i risultati dell'inferenza, identificando la parola con il punteggio di confidenza più elevato. Basandosi su questa identificazione, l'app può aggiornare l'interfaccia utente in tempo reale, registrare l'evento in una cronologia dei log e, in caso di rilevamento di parole "sensibili" (come "stop" o "off"), inviare notifiche push all'utente per avvisarlo.

Alcuni cenni agli strumenti suggeriti e adottati.

Introduzione a TensorFlow Lite

TensorFlow Lite è un framework open-source sviluppato da Google, specificamente ottimizzato per l'inferenza di modelli di machine learning su dispositivi con risorse limitate, come smartphone. Il suo obiettivo primario è abilitare l'esecuzione di modelli di intelligenza artificiale direttamente sul dispositivo (**on-device**), offrendo numerosi vantaggi:

- **Bassa Latenza:** Le inferenze avvengono localmente, eliminando la necessità di inviare dati al cloud e riducendo drasticamente i tempi di risposta. Questo aspetto è cruciale per applicazioni in tempo reale, come il riconoscimento vocale, rendendo l'app estremamente performante e reattiva.
- **Privacy:** I dati sensibili non lasciano mai il dispositivo, garantendo una maggiore protezione della privacy dell'utente.
- **Affidabilità:** L'applicazione può funzionare anche in assenza di connettività di rete, rendendola robusta in ambienti offline.

- **Dimensioni Ridotte:** I modelli TensorFlow Lite sono compressi e ottimizzati per occupare uno spazio di archiviazione minimo sul dispositivo.

Nel contesto di questa applicazione, le API di TensorFlow Lite sono state utilizzate specificamente per la **classificazione audio**.

Il Modello 'Speech Commands'

Il modello adottato in questo progetto è un modello TensorFlow Lite pre-addestrato denominato `speech_commands.tflite`. Questo modello è stato sviluppato per il riconoscimento di un set limitato di comandi vocali brevi e singoli. È stato addestrato su un dataset specifico che include parole come "up", "down", "left", "right", "on", "off", "stop", "go", oltre a categorie generiche come "silence" (silenzio) e "background" (rumore di fondo o parole non riconosciute).

Difficoltà di Classificazione con Modelli Pre-addestrati

L'adozione di un modello pre-addestrato come `speech_commands.tflite` ha notevolmente accelerato lo sviluppo dell'applicazione, consentendo di concentrarsi sull'integrazione delle funzionalità Android. Tuttavia, l'uso di un modello generico comporta alcune difficoltà :

- **Vocabolario Limitato:** Il modello è addestrato su un set fisso e ristretto di parole. Non è in grado di riconoscere termini al di fuori di questo vocabolario. Parole non previste verranno classificate come "unknown" o "background noise", o, in alcuni casi, erroneamente associate a parole conosciute con bassa confidenza.
- **Accento e Pronuncia:** Il modello è stato addestrato su un dataset specifico, presumibilmente in inglese e con accenti standard. Accenti diversi, dialetti o pronunce non standard possono ridurre significativamente la precisione del riconoscimento. Ad esempio, è stata riscontrata una maggiore difficoltà nel riconoscimento della parola "off".
- **Ambiente Acustico:** La presenza di rumore di fondo (come conversazioni, musica, traffico) può confondere il modello, portando a classificazioni errate o a un'alta confidenza per "background noise" anche in presenza di una parola valida. Il modello è sensibile all'ambiente in cui è stato addestrato. È stato spesso osservato che il modello restituisce confidenze elevate per parole non pronunciate, ma che, nella lingua italiana, sono contenute all'interno di altre parole (ad esempio, "on" e "up" possono essere "sentite" in parole italiane).
- **Variazioni del Volume e della Distanza:** Le fluttuazioni nel volume della voce o nella distanza della fonte sonora dal microfono possono influenzare negativamente le performance di classificazione.
- **Confidenza e Soglie:** Il modello produce un punteggio di confidenza per ogni categoria. La definizione di una soglia ottimale per accettare una classificazione come valida è determinante: una soglia troppo alta può causare mancate rilevazioni (falsi negativi), mentre una troppo bassa può generare molte classificazioni errate (falsi positivi). Nonostante ciò, in condizioni ambientali ottimali (es. una stanza con basso rumore di fondo), il modello può fornire inferenze con confidenze superiori al 90%.
- **Mancanza di Personalizzazione:** Non era previsto nel progetto l'addestramento ulteriormente con parole o dati specifici.

Per **adempiere al requisito di notifica in tempo reale e cronologia degli eventi**, e data la limitatezza del vocabolario del modello, sono state identificate per scopi didattici le parole "**off**" e "**stop**" come "**sensibili**". Quando il modello classifica queste parole con sufficiente confidenza, l'app non solo le registra nei log con un messaggio personalizzato rispetto alle altre classificazioni, ma invia anche una notifica push all'utente, indipendentemente dal fatto che l'app sia in **foreground** o in **background**.

Analisi Dettagliata dei File del Progetto

Di seguito viene fornita una descrizione approfondita di ogni file del progetto, illustrando il suo ruolo e le sue interazioni con gli altri componenti dell'applicazione.

1. AndroidManifest.xml

Questo file è la dichiarazione fondamentale dell'applicazione Android, definendone la struttura, i componenti principali, i permessi richiesti e le funzionalità hardware necessarie per il suo corretto funzionamento.

- **Permessi (<uses-permission>):**
 - android.permission.RECORD_AUDIO: Permesso essenziale che consente all'applicazione di accedere al microfono del dispositivo per registrare l'audio necessario alla classificazione.
 - android.permission.FOREGROUND_SERVICE: Permesso obbligatorio per le applicazioni che devono eseguire operazioni a lungo termine in background, come la classificazione audio continua, garantendo che il servizio non venga terminato dal sistema.
 - android.permission.FOREGROUND_SERVICE_MICROPHONE: questo permesso specifico dichiara che il Foreground Service utilizzerà il microfono. È una dichiarazione obbligatoria nelle ultime versioni Android più trasparente e sicura.
 - android.permission.WAKE_LOCK: Permesso utile per impedire che il dispositivo entri in modalità di sospensione profonda mentre il servizio di classificazione è attivo, assicurando un'operatività ininterrotta e una maggiore reattività, probabilmente non ottimale in termini di dispendio risorse ma utile a scopo didattico nel contesto progettuale
 - android.permission.POST_NOTIFICATIONS: necessario per consentire all'app di inviare notifiche all'utente. Questo è cruciale per gli avvisi relativi alle parole sensibili.
- **Attività (<activity>):**
 - android:name=".MainActivity": Dichiara la classe MainActivity come l'attività principale dell'applicazione.
 - android:exported="true": Indica che questa attività può essere avviata da componenti esterni all'app (ad esempio, il launcher di sistema).
 - <intent-filter>: Definisce l'attività come il punto di ingresso principale dell'app (ACTION_MAIN, CATEGORY_LAUNCHER), rendendola visibile nell'elenco delle applicazioni del dispositivo.
- **Servizio (<service>):**
 - android:name=".AudioClassificationService": Dichiara la classe AudioClassificationService come un servizio dell'applicazione.
 - android:enabled="true": Il servizio è abilitato.
 - android:exported="false": Il servizio non può essere avviato o interagito da altre applicazioni, garantendo che solo la tua app possa controllarlo.

- `android:foregroundServiceType="microphone"`: Specifica il tipo di Foreground Service. Per Android 14+ (API 34+), è obbligatorio dichiarare questo tipo se il servizio accede al microfono in background, garantendo la conformità alle ultime normative di sistema.

Interazione: L'`AndroidManifest.xml` è il punto di partenza per il sistema Android per comprendere l'app. Senza le dichiarazioni corrette in questo file, l'applicazione non avrebbe i permessi necessari per registrare l'audio o eseguire il servizio in background, non potrebbe inviare notifiche push all'utente e i suoi componenti non sarebbero riconosciuti dal sistema operativo.

2. `AudioClassificationHelper.java`

Questa classe funge da helper per la logica di classificazione audio. Incapsula l'interazione con le librerie TensorFlow Lite per l'elaborazione audio e la gestione dell'hardware del microfono, occupandosi dell'inferenza.

- **Responsabilità Principali:**
- **Inizializzazione del Classificatore:** Carica il modello TensorFlow Lite (`speech_commands.tflite`) e configura l'oggetto `AudioClassifier` con le opzioni desiderate, come il numero di thread per l'inferenza e la soglia di confidenza predefinita.
- **Acquisizione Audio:** Configura e gestisce l'oggetto `AudioRecord`, l'API Android per l'acquisizione di dati audio dal microfono.
- **Esecuzione dell'Inferenza:** Utilizza un `ScheduledExecutorService` per leggere periodicamente i dati audio dal microfono, caricarli nel `TensorAudio` (il formato di input per il modello) e passarli all'`AudioClassifier` per eseguire l'inferenza.
- **Gestione dello Stato:** Mantiene lo stato corrente della registrazione (`isRecording`) e dell'inizializzazione del classificatore (`isClassifierInitialized`), necessario quando l'app presenta più thread.
- **Rilascio Risorse:** Si occupa di fermare la registrazione audio e rilasciare tutte le risorse hardware e TensorFlow Lite (`AudioRecord`, `AudioClassifier`, `TensorAudio`). Necessario per prevenire ottimizzare le risorse e assicurare che il microfono sia nuovamente disponibile per altre applicazioni, contribuendo a mantenere l'app responsiva e a non assorbire troppe risorse dal dispositivo.
- **Callback dei Risultati/Errori:** Definisce un'interfaccia `ClassifierListener` che viene implementata dalla classe chiamante (nel nostro caso il servizio, `AudioClassificationService`). Attraverso questa interfaccia, l'helper comunica i risultati della classificazione (una lista di `Category` con etichette e punteggi di confidenza, e il tempo di inferenza) o eventuali errori che si verificano durante il processo.
- **Variabili Chiave**
Per una maggiore leggibilità e manutenzione del codice ho dichiarato alcune variabili che aiutino allo scopo:
- `MODEL_NAME`: Il nome del file del modello TensorFlow Lite (`speech_commands.tflite`).

- **CLASSIFIER_INTERVAL_MS:** L'intervallo di tempo tra un'inferenza e l'altra (attualmente 200 ms, corrispondenti a 5 inferenze al secondo), bilanciando responsività e consumo di risorse.
- **DEFAULT_CLASSIFIER_THRESHOLD:** La soglia di confidenza predefinita (0.80f) utilizzata per configurare il classificatore.
- **AudioClassifier:** L'oggetto TensorFlow Lite che esegue l'operazione di classificazione.
- **TensorAudio:** L'oggetto che funge da contenitore per i dati audio pre-elaborati, pronti per essere forniti al modello.
- **AudioRecord:** L'API Android utilizzata per l'acquisizione diretta dell'audio dal microfono del dispositivo.

Interazione: `AudioClassificationHelper` è utilizzato esclusivamente da `AudioClassificationService`. Il servizio è responsabile di inizializzare, avviare e fermare la classificazione tramite questo helper, ricevendo i risultati e gli errori attraverso l'interfaccia `ClassifierListener`.

3. `AudioClassificationService.java`

Questa classe estende `android.app.Service`, il che le consente di eseguire operazioni in background senza un'interfaccia utente diretta. Funge da collegamento tra la logica di classificazione (`AudioClassificationHelper`) e l'interfaccia utente (`MainActivity`), gestendo anche le notifiche di sistema.

- **Responsabilità Principali:**
- **Gestione del Ciclo di Vita del Servizio:** Controlla l'avvio (`onStartCommand`) e l'arresto (`onDestroy`, `stopSelf`) del servizio in base agli `Intent` ricevuti dalla `MainActivity` o dal sistema.
- **Servizio in Foreground:** Avvia il servizio come "foreground service" (`startForeground`). Questa è una pratica fondamentale in Android per le operazioni continue in background, in quanto rende il servizio meno propenso a essere terminato dal sistema (ad esempio, per mancanza di memoria) e mostra una notifica persistente all'utente, indicando che l'applicazione sta utilizzando il microfono.
- **Comunicazione con `AudioClassificationHelper`:** Inizializza, avvia e ferma l'istanza di `AudioClassificationHelper`. Implementa l'interfaccia `AudioClassificationHelper.ClassifierListener` per ricevere i risultati delle inferenze (classificazioni) e gli eventuali errori direttamente dall'helper.
- **Gestione dei Log in Background:** Mantiene una `LinkedList` denominata `backgroundLogEntries`. Questa lista memorizza una cronologia delle classificazioni più recenti che superano una certa soglia di confidenza, includendo anche le classificazioni di "rumore di fondo" e "silenzio". Questa cronologia è fondamentale per fornire dati alla `MainActivity` quando questa torna in foreground, in quanto durante l'attività in background la main activity non riesce ad aggiornare i log in tempo reale che verranno altrimenti persi.
- **Notifiche di Sistema:**
- Crea e configura due canali di notifica (`NotificationChannel`): uno per lo stato generale del servizio (`AudioClassifierChannel`) e uno dedicato agli avvisi per le parole sensibili (`SensitiveWordsChannel`). Questo permette agli utenti di gestire le impostazioni delle notifiche in modo personalizzato.

- Aggiorna la notifica in foreground con i risultati di classificazione in tempo reale, mantenendo l'utente informato sull'attività del servizio.
- Invia notifiche push specifiche (`showSensitiveWordNotification`) quando vengono rilevate parole considerate "sensibili" (come "stop" o "off") con una confidenza sufficientemente alta.
- **Comunicazione con MainActivity (Broadcasts):**
 - Invia broadcast locali (`LocalBroadcastManager`) alla MainActivity per comunicare vari eventi:
 - `ACTION_CLASSIFICATION_RESULT`: I risultati di ogni inferenza, inclusi etichette, confidenze e tempo di elaborazione.
 - `ACTION_CLASSIFICATION_ERROR`: Eventuali errori critici rilevati durante il processo di classificazione.
 - `ACTION_SERVICE_INITIALIZED`: Segnala che il servizio è stato avviato con successo e il classificatore è pronto all'uso.
 - `ACTION_SERVICE_STOPPED`: Notifica che il servizio è stato interrotto.
 - `ACTION_LOG_HISTORY_RESPONSE`: Invia la cronologia completa dei log in background in risposta a una richiesta della MainActivity.
 - Riceve Intent dalla MainActivity per controllare il proprio stato:
 - `ACTION_START_CLASSIFICATION`: Richiesta per avviare il processo di classificazione audio.
 - `ACTION_STOP_CLASSIFICATION`: Richiesta per interrompere il processo di classificazione.
 - `ACTION_REQUEST_LOG_HISTORY`: Richiesta di ottenere la cronologia dei log accumulati.
- **Variabili Chiave:**
 - `SENSITIVE_WORDS`: Una lista predefinita di stringhe che identificano le parole considerate sensibili.
 - `SENSITIVE_WORD_THRESHOLD`: La soglia di confidenza che una parola sensibile deve superare per attivare la notifica push.
 - `BACKGROUND_LOG_THRESHOLD`: La soglia di confidenza per includere una classificazione nella cronologia dei log in background.
 - `MAX_BACKGROUND_LOG_ENTRIES`: Il numero massimo di voci di log che il servizio mantiene nella sua cronologia interna.

Interazione: `AudioClassificationService` permette all'app di lavorare in background. Avvia e gestisce l'`AudioClassificationHelper`. Comunica bidirezionalmente con la MainActivity tramite Intent e `LocalBroadcastManager` per aggiornare l'interfaccia utente e ricevere comandi, e interagisce con il sistema Android per la gestione delle notifiche.

4. `ClassificationLogEntry.java`

Questa è una classe che descrive il modello di dati per incapsulare un singolo evento di classificazione audio.

- **Responsabilità Principali:**
- **Modello Dati:** Contiene tre campi pubblici:
 - `label`: Una String che rappresenta la parola o la categoria classificata (es. "stop", "silence", "background_noise_").

- **confidence:** Un float che indica il punteggio di confidenza (probabilità) della classificazione.
- **timestamp:** Un long che registra il momento esatto (in millisecondi) in cui è avvenuta l'inferenza.
-
- **Implementazione Parcelable:** Implementa l'interfaccia `Parcelable` di Android. Questa implementazione è cruciale perché consente di serializzare e deserializzare in modo efficiente gli oggetti `ClassificationLogEntry`. Ciò permette di passare facilmente e rapidamente liste di questi oggetti tra diversi componenti dell'applicazione (nel nostro caso, dal `AudioClassificationService` alla `MainActivity`) tramite gli `Intent` (`putParcelableArrayListExtra`).

Interazione: `ClassificationLogEntry` è utilizzata principalmente dall'`AudioClassificationService` per popolare la sua lista `backgroundLogEntries` (la cronologia interna dei log). Successivamente, il servizio invia queste liste di oggetti `ClassificationLogEntry` alla `MainActivity` tramite `Intent broadcast`. La `MainActivity`, a sua volta, utilizza questi oggetti per popolare la sua `recentLogEntriesList` e visualizzare i dati nel log scorrevole dell'interfaccia utente.

5. MainActivity.java

Questa è l'attività principale dell'applicazione, che gestisce l'interfaccia utente (UI) e l'interazione diretta con l'utente.

- **Responsabilità Principali:**
- **Gestione UI:** Inizializza tutti gli elementi dell'interfaccia utente (pulsanti, `TextView`, `ScrollView`) definiti nel file di layout `activity_main.xml`. Si occupa di aggiornare questi elementi per riflettere lo stato dell'applicazione e i risultati della classificazione.
- **Richiesta Permessi:** Controlla e richiede in fase di runtime i permessi necessari all'applicazione (`RECORD_AUDIO` per il microfono e `POST_NOTIFICATIONS` per le notifiche) sia all'avvio dell'attività che quando l'utente tenta di avviare la classificazione.
- **Avvio/Arresto Servizio:** Gestisce gli eventi di click sul pulsante "Avvia/Interrompi Classificazione". In base allo stato attuale dell'applicazione, invia gli `Intent` appropriati (`ACTION_START_CLASSIFICATION` o `ACTION_STOP_CLASSIFICATION`) all'`AudioClassificationService` per controllare il processo di classificazione.
- **Ricezione Broadcasts:** Registra un `BroadcastReceiver` (`classificationReceiver`) per ascoltare attivamente i broadcast locali inviati dall'`AudioClassificationService`. Questo permette alla `MainActivity` di ricevere aggiornamenti in tempo reale sui risultati della classificazione, eventuali errori o cambiamenti nello stato del servizio.
- **Aggiornamento UI in Tempo Reale:**
- `handleClassificationResults()`: Questo metodo riceve i risultati di classificazione dal servizio. **Seleziona l'inferenza con la confidenza più alta** ordinando la lista dei risultati e prendendo il primo elemento. Implementa una logica di **debouncing** per la `displayTextView` principale, evitando aggiornamenti troppo rapidi. L'UI viene aggiornata solo se il comando è cambiato, la confidenza è significativamente aumentata, o se è trascorso un certo tempo per gli stati di "silenzio" o "rumore di fondo".
- `updateRecentLog()`: Aggiunge le classificazioni pertinenti (quelle sopra una certa soglia di confidenza o le categorie "rumore di fondo"/"silenzio") alla `recentLogEntriesList`. Questo

metodo include una logica di **debouncing per la lista dei log**, raggruppando le voci identiche che si verificano in un breve arco di tempo (UI_RECENT_LOG_GROUPING_TIME_MS) e aggiornando la confidenza con il valore più alto rilevato.

- `updateRecentInferencesTextView()`: Aggiorna la `recentInferencesTextView` (la `TextView` all'interno della `ScrollView` che mostra la cronologia dei log) con le voci più recenti della `recentLogEntriesList`.
- **Gestione Stato Servizio**: Aggiorna lo stato del pulsante "Avvia/Interrompi Classificazione" e il testo principale (`displayTextView`) in base ai broadcast di stato ricevuti dal servizio, mantenendo l'UI sincronizzata.
- **Richiesta Cronologia Log**: Invia un `Intent` al servizio per richiedere la cronologia dei log accumulati in background, specialmente quando l'attività viene ripresa o il servizio viene inizializzato.
- **Variabili Chiave**:
 - `UI_DISPLAY_WORD_THRESHOLD`: Soglia di confidenza per visualizzare una parola nella `displayTextView` principale.
 - `UI_RECENT_LOG_THRESHOLD`: Soglia di confidenza per includere una parola nel log scorrevole.
 - `KNOWN_COMMANDS`: Una lista di stringhe che rappresenta i comandi che il modello è addestrato a riconoscere, visualizzata nell'interfaccia utente.
 - `SENSITIVE_WORDS`: Una lista di stringhe che identifica le parole considerate sensibili per la logica interna dell'app.
 - `RecentLogEntry`: Una classe interna statica che modella una singola voce nel log recente visualizzato nell'UI.

Interazione: La `MainActivity` è l'interfaccia utente dell'applicazione. Invia comandi all'`AudioClassificationService` e riceve da esso i dati di classificazione e gli aggiornamenti di stato per visualizzarli all'utente in modo dinamico e responsivo.

6. `activity_main.xml`

Questo file XML definisce la struttura e il layout visivo dell'interfaccia utente per la `MainActivity`.

- **Struttura:** Il layout principale è un `LinearLayout` con orientamento verticale.
- **Componenti UI:**
 - `TextView (@+id/display_text_view)`: Posizionata in alto, questa `TextView` è destinata a mostrare il comando vocale rilevato più di recente e lo stato attuale del classificatore. È formattata con testo grande e in grassetto per essere ben visibile.
 - `Button (@+id/record_button)`: Il pulsante principale dell'interfaccia, utilizzato dall'utente per avviare o interrompere il processo di classificazione audio. È stilizzato con un colore di sfondo (`backgroundTint`), padding, angoli arrotondati (`app:cornerRadius`) ed elevazione (`elevation`) per un aspetto moderno e cliccabile.
 - `TextView (@+id/known_commands_list_text_view)`: Questa `TextView` elenca i comandi vocali che il modello `speech_commands.tflite` è in grado di riconoscere, fornendo all'utente un riferimento rapido.

- `ScrollView (@+id/log_scroll_view)`: Un contenitore scorrevole che avvolge la `recent_inferences_text_view`. Questo permette alla cronologia dei log di estendersi oltre le dimensioni dello schermo e di essere consultabile tramite scorrimento.
- `TextView (@+id/recent_inferences_text_view)`: All'interno della `ScrollView`, questa `TextView` visualizza la cronologia dettagliata delle classificazioni audio recenti. È formattata con un font monospace per una migliore leggibilità dei log.
- **Design**: L'utilizzo di `androidx.cardview.widget.CardView` per raggruppare i diversi blocchi di informazione (display principale, comandi noti, log eventi) conferisce all'interfaccia un aspetto moderno, pulito e ben organizzato. I colori e le dimensioni del testo sono definiti per garantire una buona leggibilità e un contrasto adeguato.

Interazione: Questo file XML viene gestito dalla `MainActivity` (`setContentView(R.layout.activity_main)`). Successivamente, la `MainActivity` accede ai vari elementi dell'interfaccia utente tramite i loro ID (`findViewById()`) per manipolarli, aggiornarne il contenuto e rispondere alle interazioni dell'utente.

Conclusioni

Il progetto **Keyword Spotting** soddisfa tutti i requisiti minimi prefissati: integra e utilizza un modello TensorFlow Lite (`.tflite`) funzionante in locale (posizionato nella cartella `assets` del progetto), gestisce correttamente tutti i permessi necessari come specificato nell'`AndroidManifest.xml`, cattura l'audio in modo affidabile sia in background che in foreground, e completa l'inferenza delle parole riconosciute sia in back che in foreground. L'interfaccia utente è intuitiva, fornisce un report dettagliato degli ultimi log e include notifiche push per le parole sensibili, migliorando l'esperienza utente e la consapevolezza dello stato dell'app.